Pack 03 · Inside the serving engine

Batching, scheduling and paged KV memory: how vLLM-style engines keep the GPU busy without breaking latency. Lessons 19, 26, 28, 30, 32–41, 44, 45. Answer each question out loud in about a minute, then check the card.

Q1 · Static vs continuous batching PRIORITY

Lessons: 28, 32, 33, 34

> **INTERVIEWER**
>
> A static batch of 4 slots gets replies of 50, 150, 400 and 800 tokens at about 5 ms per step. What share of the work is useful, when does the 50-token user get their answer, and when is static batching actually fine?

60-second answer

- A static batch runs until its **longest** member finishes: capacity = 4 slots × 800 steps = 3,200 slot-steps, useful = 1,400, so **≈ 44%**.
- The 50-token user waits for all 800 steps: **4 s** instead of 0.25 s.
- **Iteration-level (continuous) batching** decides after every step: finished requests leave and queued ones join immediately.
- Static is fine when outputs are **uniform**, like a nightly job emitting one token per review; useful share ≈ mean length ÷ max length.

Key numbers: static useful share = Σ lengths ÷ (slots × longest).

Likely follow-up: "Why is continuous batching the single biggest serving win?" It keeps the batch full, so every weight read serves as many tokens as possible, and short requests don't wait for long ones.

> **TRAP**
>
> I didn't know what to divide by: capacity is slots × the longest reply, not the sum of replies.

Q2 · A ragged step: pad it or not? PRIORITY

Lessons: 33

> **INTERVIEWER**
>
> On one step, two requests decode one token each and a new request brings a 4-token prompt. A junior suggests padding all three to 4 tokens. What's wasted, and how does Orca's selective batching avoid padding?

60-second answer

- Padding gives 12 slots for 6 real tokens: **50% wasted**.
- Selective batching **flattens all 6 tokens into one batch** for the token-wise layers: QKV and output projections, the MLP and the norms. Those treat each token independently.
- **Attention is split per request**, because each token must attend only to **its own request's history** (its KV cache) under a causal mask.

Key numbers: padded waste = 1 − real tokens ÷ (requests × longest).

Likely follow-up: "How does the attention kernel handle different lengths?" It reads each request's KV through its own block table (PagedAttention), so no padding is needed there either.

> **TRAP**
> I listed only the MLP as token-wise and didn't say why attention must be split.

Q3 · The scheduler is now on the clock PRIORITY

Lessons: 33

> **INTERVIEWER**
> With iteration-level scheduling, each GPU step takes 5 ms and the Python scheduler spends 2 ms per step deciding who runs, with the GPU idle meanwhile. What fraction is wasted, and why was this invisible before?

60-second answer

- Cycle = 5 + 2 = 7 ms, so **2 ÷ 7 ≈ 29%** of GPU time is wasted and throughput drops to **≈ 71%**.
- Under request-level scheduling, the scheduler ran once per request, not once per step, so its cost was negligible.
- Fixes: **overlap** scheduling for step n + 1 with GPU step n (async scheduling), make the scheduler faster, and use CUDA graphs to cut launch time.

Key numbers: waste = CPU time ÷ (GPU time + CPU time).

Likely follow-up: "How would you spot it?" GPU-idle gaps between kernels in a profiler trace, and throughput that doesn't rise with batch size.

> **TRAP**
> I divided 2 ÷ 5. The denominator is the whole cycle.

Q4 · Decode first, and sizing the token budget PRIORITY

Lessons: 34, 35, 45

> **INTERVIEWER**
>
> A teammate wants new prompts scheduled before running decodes "to cut TTFT". And for Llama 3.1 8B on an H100 (≈ 4.8 ms floor, then ≈ 16 μs per token), with 32 users decoding and a 20 ms inter-token target, what's the largest prefill chunk per step?

60-second answer

- Prompts first would **stall every user mid-answer**; decodes first protects TPOT and the KV already invested in running requests.
- Step time ≈ **max(4.8 ms of weight reads, tokens × 16 μs of math)**: the two overlap, so it's a max, not a sum.
- 20 ms ÷ 16 μs = 1,250 tokens per step; minus 32 decode tokens = 1,218, rounded **down to a multiple of 128: 1,152**.
- That's **chunked prefill**: long prompts are split across steps so no step blows the inter-token budget.

Key numbers: budget ≈ ITL target ÷ per-token cost; prompt room = budget − running decodes; steps for a prompt = ⌈prompt ÷ room⌉.

Likely follow-up: "A 6,000-token prompt with 150 decodes and a 25 ms target?" 25 ms ÷ 16 μs ≈ 1,560; minus 150 = 1,410 per step; ⌈6,000 ÷ 1,410⌉ = 5 steps.

> **TRAP**
>
> I once added the math on top of 4.8 ms ("microseconds") instead of taking the max. Let units decide: ms ÷ μs gives a token count.

Q5 · Which gate is closed? PRIORITY

Lessons: 30, 35

> **INTERVIEWER**
>
> num_requests_running is stuck at exactly 64 for an hour, num_requests_waiting climbs past 200, KV cache usage is 40%, and steps are well under the latency target. What's blocking new requests?

60-second answer

- A request needs room in **both** budgets: sequences running (**max_num_seqs**) and tokens this step (**max_num_batched_tokens**), plus free KV blocks.
- Running pinned at exactly 64, KV at 40% and fast steps all point to **max_num_seqs = 64**.
- Raise it, then re-check KV usage, preemptions and TPOT at the new level.

Key numbers: admitted only if running < max_num_seqs, tokens this step ≤ budget, and KV blocks are free.

Likely follow-up: "256 requests decoding, max_num_seqs 256, budget 8,192, and a 500-token prompt arrives?" The tokens fit (256 + 500 < 8,192) but the sequence budget is full, so it waits in `num_requests_waiting` until one request finishes.

> **TRAP**
>
> Check each budget separately. I didn't know where to start on the 256-sequence case.

Q6 · Contiguous slabs vs paged blocks, and the block size PRIORITY

Lessons: 19, 28, 37, 38

> **INTERVIEWER**
>
> An engine reserves one contiguous 8,192-token KV slab per request. The pool is 50 GB and requests average 1,000 tokens (Llama 3.1 8B). How many fit, how much is wasted, what do paged blocks change, and why use 16-token blocks?

60-second answer

- Slab = 8,192 × 128 KiB = **1 GiB**, so ≈ **46 requests**, each only ≈ **12%** used. Storing only used tokens fits ≈ **380**.
- Contiguous wastes three ways: **reserved** (space held for tokens not generated yet), **internal fragmentation** (the request stops short and the rest is never used) and **external fragmentation** (free gaps too small for a new slab). Compaction would copy gigabytes of live KV, stalling everyone.
- **Paged KV**: fixed blocks handed out as tokens arrive, tracked by a per-request block table. Waste is at most one partial block per request.
- Block size: 1 token means huge tables and scattered reads; 256 means up to 255 wasted tokens per request. **16** keeps waste around 8 tokens while reads stay efficient.

Key numbers: blocks = ⌈tokens ÷ 16⌉; token t lives in table[t // 16], slot t % 16.

Likely follow-up: "What's the catch of paging?" The engine admits requests on their current size, so if many grow at once the pool runs dry mid-generation and it must preempt.

> **TRAP**
>
> I dropped the 2 bytes for BF16 and divided upside down; a request count must come out bigger than 1. And small blocks cost bookkeeping and scattered reads, not "recomputation".

Q7 · "PagedAttention must approximate, and its kernel is slower" PRIORITY

Lessons: 39, 40

> **INTERVIEWER**
>
> An interviewer says PagedAttention reads scattered keys and values, so it must approximate attention, and its kernel is 20–26% slower. How can vLLM claim 2–4× higher throughput? And what does paging do for n = 4 samples of one 1,000-token prompt?

60-second answer

- It's **exact**: the same math, just reading K and V through a block table. The slowdown comes from that indirection inside the kernel.
- Paging wastes almost no memory, so **far more requests fit**, and bigger batches share each memory-bound weight read. Attention is a small part of the step, so the server gets much faster.
- Sharing: the prompt needs $\lceil 1{,}000 \div 16 \rceil = 63$ **63 blocks** shared, versus 252 for four copies.
- The last block is partial, so the first write by each sample triggers **copy-on-write**; the last writer can write in place. That's 3 copies, **66 blocks** in total.

Key numbers: copies for n samples sharing a partial block = $n - 1$.

Likely follow-up: "Where else does block sharing pay off?" Prefix caching across requests (pack 05) and beam search.

> **TRAP**
>
> I blamed the kernel slowdown on allocation and eviction. Those are scheduler work; the kernel cost is block-table indirection.

Q8 · Preemption: who goes, how, and when to worry PRIORITY

Lessons: 36, 40, 41

> **INTERVIEWER**
>
> num_preemptions_total climbs every minute and p99 latency spikes; requests average about 2,000 tokens. Who gets evicted and how? A teammate suggests evicting the biggest request instead. And is a KV cache pinned at 100% always a problem?

60-second answer

- One request holds 2,000 × 128 KiB ≈ **262 MB**.
- Evict the **newest** request (least progress lost), and evict it **whole**: attention needs the full history. Evicting the biggest throws away the most work.
- Two ways: **recompute** (re-prefill later, costs compute) or **swap** to CPU memory (costs PCIe transfer). The real fix is admitting less (max_num_seqs) or adding KV memory (fp8 KV, more GPUs).
- **100% usage with zero preemptions is healthy**: freed blocks stay cached for reuse. The pressure signal is **rising preemptions**, which users feel as mid-stream stalls.

Key numbers: steps until the pool runs dry = free blocks × 16 ÷ running requests (2,000 × 16 ÷ 250 = 128 steps ≈ 2.6 s at 20 ms).

Likely follow-up: "Why didn't the scheduler see it coming?" It admits on each request's current size, not its final size.

> **TRAP**
>
> I answered with queue order (who starts) instead of eviction (who leaves), and called queue length the KV-pressure signal. It's preemptions.

Q9 · Cancellation and fairness across customers PRIORITY

Lessons: 36

> **INTERVIEWER**
>
> A user closes the tab at token 300 of a reply capped at 1,000 tokens and the server never notices. What does it cost? Separately, client A sends 90% of traffic and small client B has terrible p99 TTFT. The PM proposes a hard per-client rate limit. What would you do instead?

60-second answer

- No cancellation: **700 tokens** generated for nobody, and the request grows to 1,000 × 128 KiB ≈ **131 MB** of KV. Abort on disconnect to free its slot and KV at once.
- A hard rate limit **rejects work and idles the GPU** even when nobody else is waiting.
- Use a **virtual token counter (VTC)**: count each client's weighted served tokens and admit from the client with the lowest count. It **reorders admission, never rejects**.
- It's work-conserving: when B goes quiet, A gets the whole GPU.

Key numbers: wasted tokens = max_tokens − tokens when the user left.

Likely follow-up: "What stops B from saving up credit while idle?" When B returns, its counter is lifted to the current minimum, so it can't burst.

> **TRAP**
>
> I called VTC "rate limiting". It never rejects anything. And on cancellation I described the fix instead of computing the waste asked for.

Q10 · Why RAG traffic slows the chat users PRIORITY

Lessons: 26, 44, 45

> **INTERVIEWER**
>
> Chat (500-token prompts) and RAG (6,000-token prompts) share one H100. Since RAG was added, chat streams more slowly although chat traffic is unchanged, yet p99 inter-token latency looks the same as before. Explain both.

60-second answer

- **Prefill interference:** each long RAG prefill takes a whole step (≈ 130 ms for 8k tokens), pausing every chat stream already decoding.
- **Memory:** RAG requests carry big KV caches, so every shared decode step reads more bytes.
- **p99 can be blind**: 4 stalls among ≈ 3,940 token gaps is 0.1%, below the 1% that p99 ignores. Report max, p99.9, a stall count, or the gap histogram.
- Fix: **chunked prefill** (Q4), or separate prefill and decode onto different GPUs (disaggregation).

Key numbers: a percentile p only sees events more frequent than $1 - p$.

Likely follow-up: "Why not just look at TPOT?" TPOT averages the gaps, so a few long freezes vanish.

> **TRAP**
>
> I said chat requests queue. They're already decoding and get paused by prefill. I also forgot the memory reason.

Build it yourself

- `exercises/18_19_paged_kv.py`: simulates reserving the max length versus paged blocks with preemption. It shows paging runs 4.5× more requests at once at the cost of evictions. Run `uv run exercises/18_19_paged_kv.py`.
- `exercises/32_36_batching_sim.py`: a static versus continuous batching simulator (steps, useful share, average finish time). Run `uv run exercises/32_36_batching_sim.py`.
- `exercises/37_41_block_manager.py`: a block manager with a free queue, reference counts and copy-on-write (`allocate`, `fork`, `append`, `free`). Run `uv run exercises/37_41_block_manager.py`.